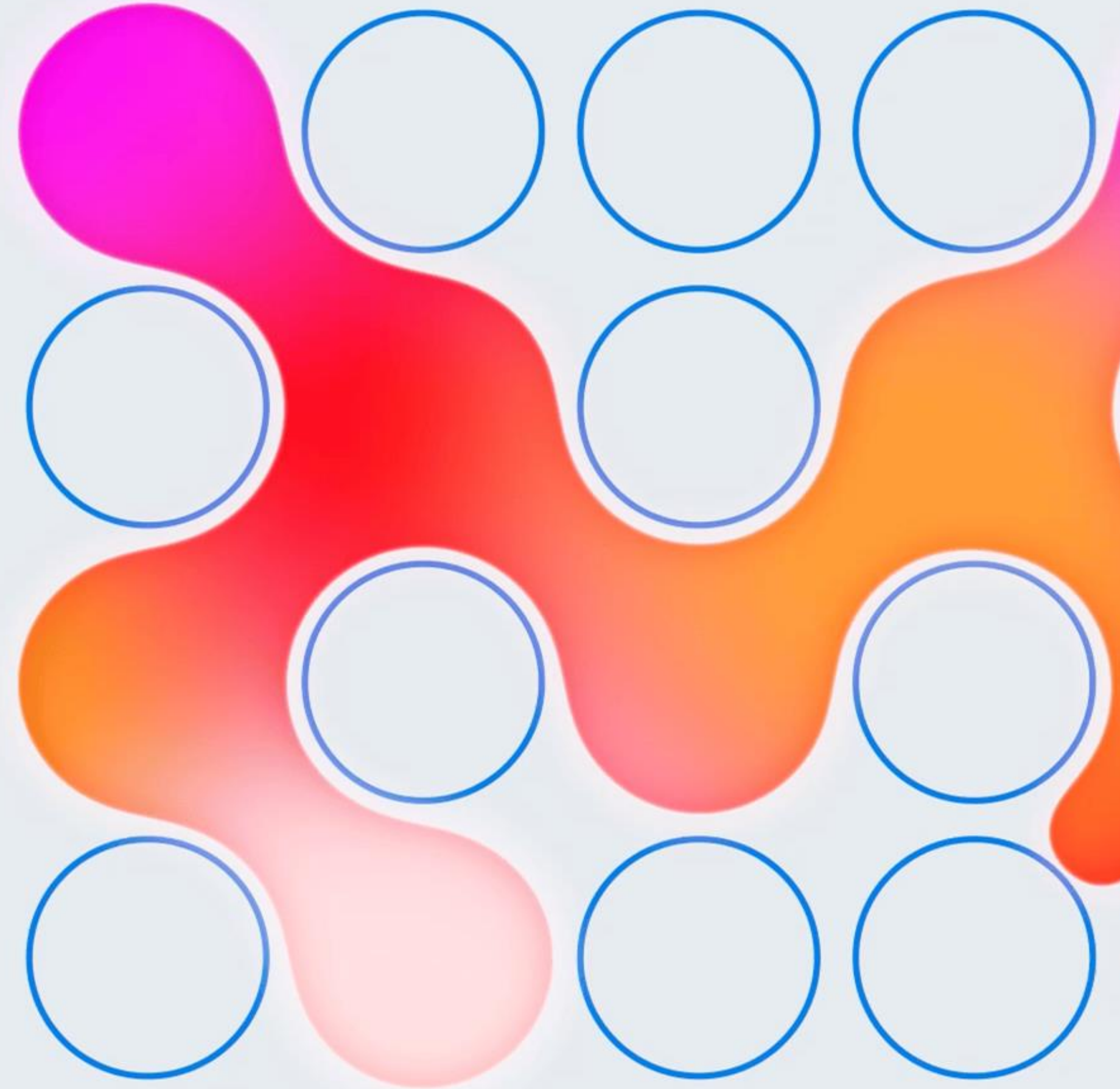




Building Agentic Systems on AWS

Workshop: Multi-Agent Architectures



Presenters

Dr. James Bland

WW Tech Lead, Data & AI,
Amazon Web Services (AWS)



Mike Brugnoni

Sr. Solutions Architect, AWS



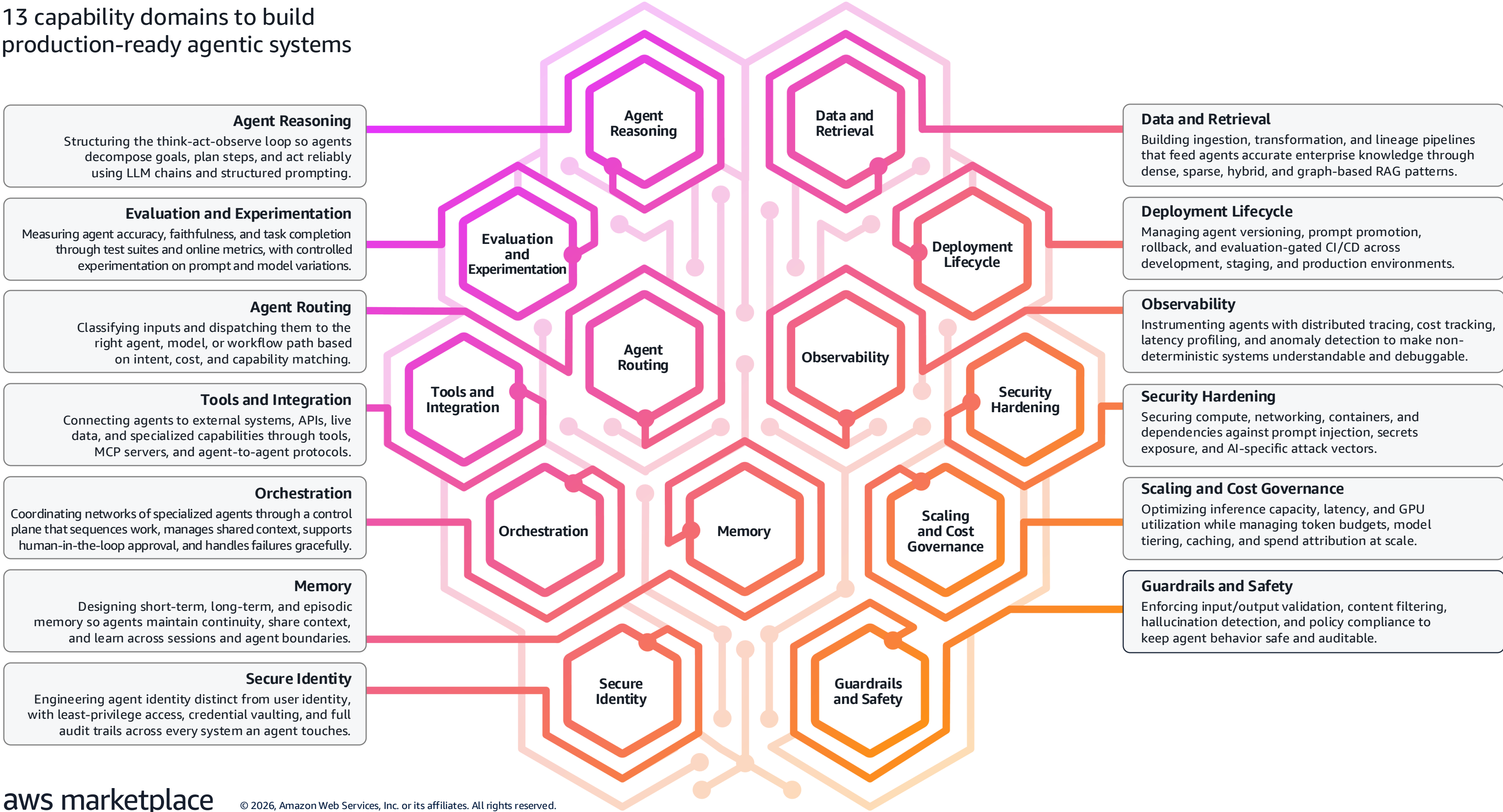
Agenda

- Why multi-agent?
- Anatomy of a multi-agent system
- Orchestration patterns
- Non-determinism at scale
- Shared context, state management, and handoff design
- MCP and A2A protocols
- Security, resilience, observability, and the DevOps Companion
- Demo
- Next steps



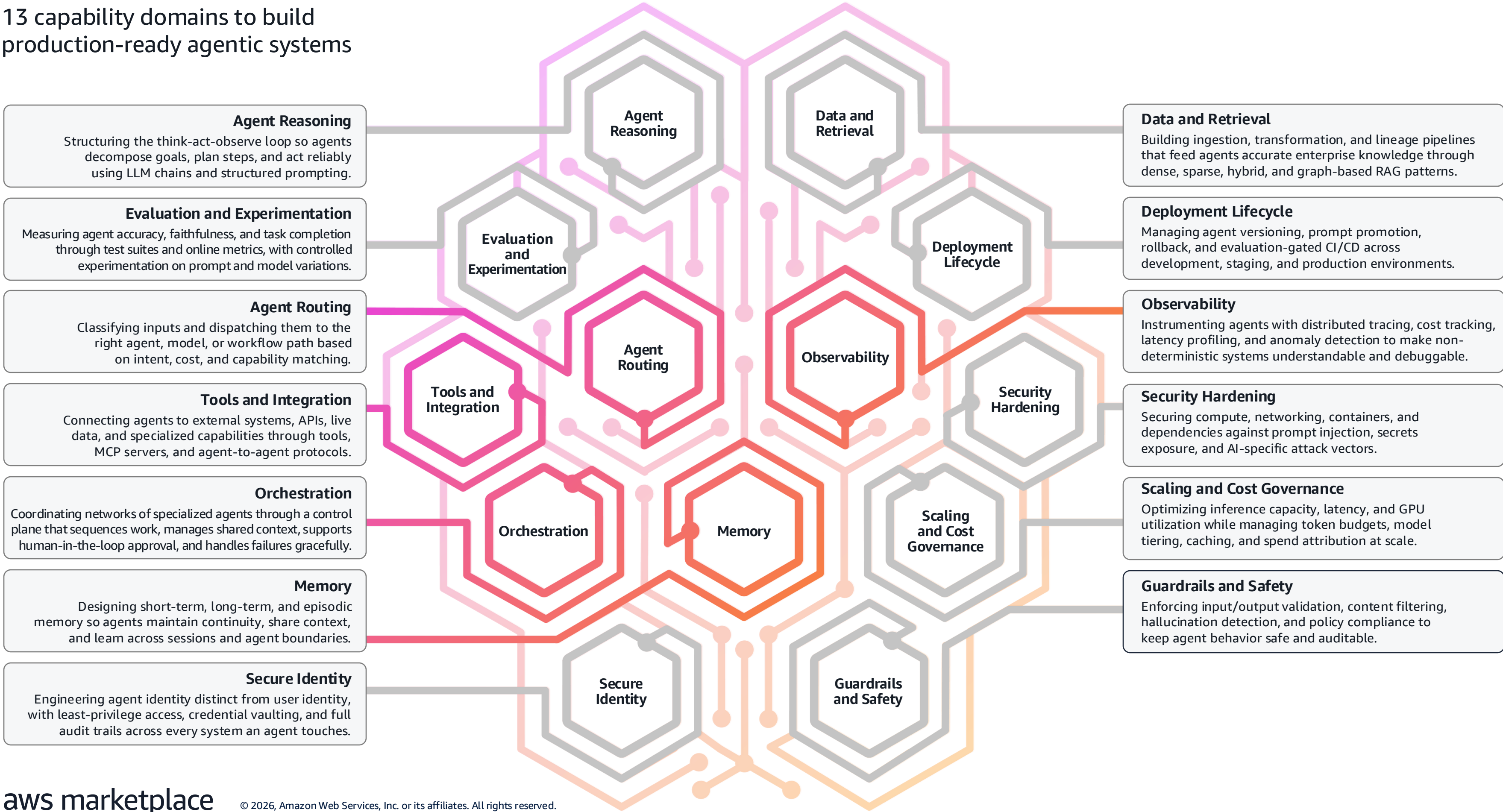
The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



Agentic AI tools and capabilities in AWS Marketplace

AGENT REASONING

Claude

cohere

Hugging Face

LangChain

Dify

Progress Agentic RAG

composable

Amazon Bedrock

STRANDS AGENTS

EVALUATION & EXPERIMENTATION

LangChain

Langfuse

vellum

Weights & Biases

comet LLM

Patronus AI

deepchecks.

Amazon Bedrock AgentCore (Evaluations)

Amazon SageMaker AI

Amazon CloudWatch

AGENT ROUTING

LangChain

AWS Step Functions

Amazon EventBridge

Amazon Bedrock

TOOLS & INTEGRATION

algolia

mapbox

Deepgram

Kong

Konnect

Tyk

TAILWINDS

portkey

SMARTBEAR.

krakenD

AWS Lambda

Amazon API Gateway

Amazon Bedrock AgentCore (Gateway)

ORCHESTRATION

Temporal

orkes

PREFECT

ASTRONJMER

AWS Step Functions

Amazon Bedrock Agents

Amazon MWA

MEMORY

zilliz

redis

elastic

TiDB Cloud

MongoDB Atlas

Weaviate

yugabyteDB

Pinecone

Amazon OpenSearch

Amazon MemoryDB

Amazon Bedrock AgentCore (Memory)

SECURE IDENTITY

Ping

okta

CYBERARK

Amazon Cognito

AWS Secrets Manager

Amazon Bedrock AgentCore (Identity)

DATA & RETRIEVAL

CONFLUENT

databricks

Fivetran

Collibra

Alation

Bigeye

MONTE CARLO

boomi

Amazon Bedrock Knowledge Bases

Amazon S3

AWS Glue

DEPLOYMENT LIFECYCLE

Pulumi

GitLab

Vercel

LaunchDarkly

Vault

KNIME

AWS CodePipeline

Amazon SageMaker AI

AWS CDK

OBSERVABILITY

DATADOG

honeycomb.io

arize

LangWatch

True Watch

INSTANA

logz.io

Amazon CloudWatch

AWS X-Ray

AWS Distro for OpenTelemetry

SECURITY HARDENING

snyk

Protect AI

Prompt

tailscale

Amazon GuardDuty

AWS Security Hub

Amazon Inspector

SCALING & COST GOVERNANCE

ClickHouse

NVIDIA AI ENTERPRISE

Modal

cast.ai

baseten

anyscale

hydrolix

StormForge

Amazon EKS

AWS Fargate

Amazon Bedrock AgentCore (Runtime)

GUARDRAILS & SAFETY

Claude

Guardrails AI

Arthur

Amazon Bedrock Guardrails

Amazon Bedrock AgentCore (Policy)

2,800+

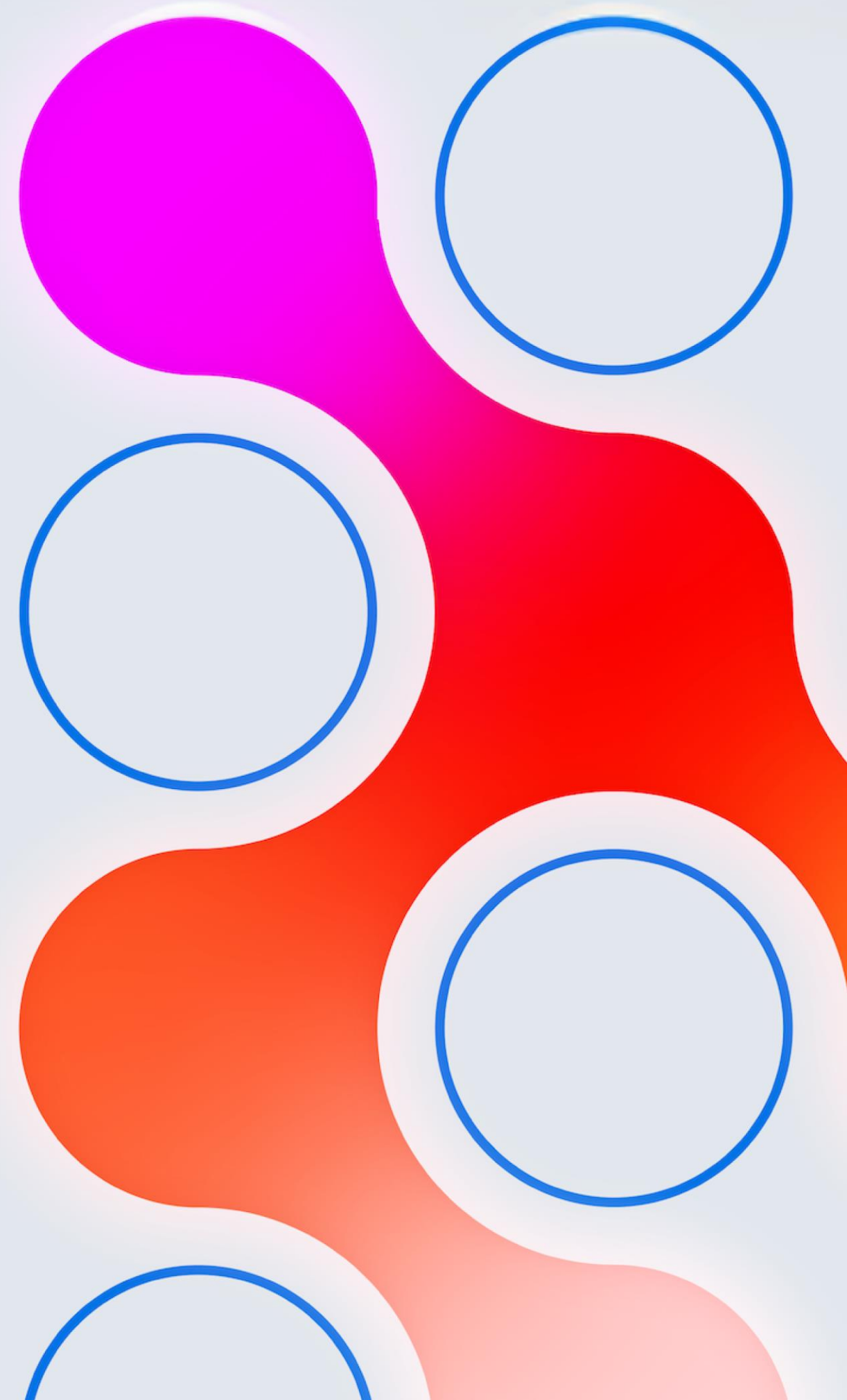
Agentic AI tools and capabilities

To help you build, deploy, and operate agentic systems on AWS — available through your AWS account.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

6

Why multi-agent? The case for collaborative AI



When single agents break down

Context saturation

Complex long-horizon tasks overflow a single context window. The agent forgets earlier work but does not know it has.

Task specialization

One agent trying to excel at security, Infrastructure-as-Code (IaC), and CI/CD simultaneously produces mediocrity across the board rather than excellence in any domain.

Latency and parallelism

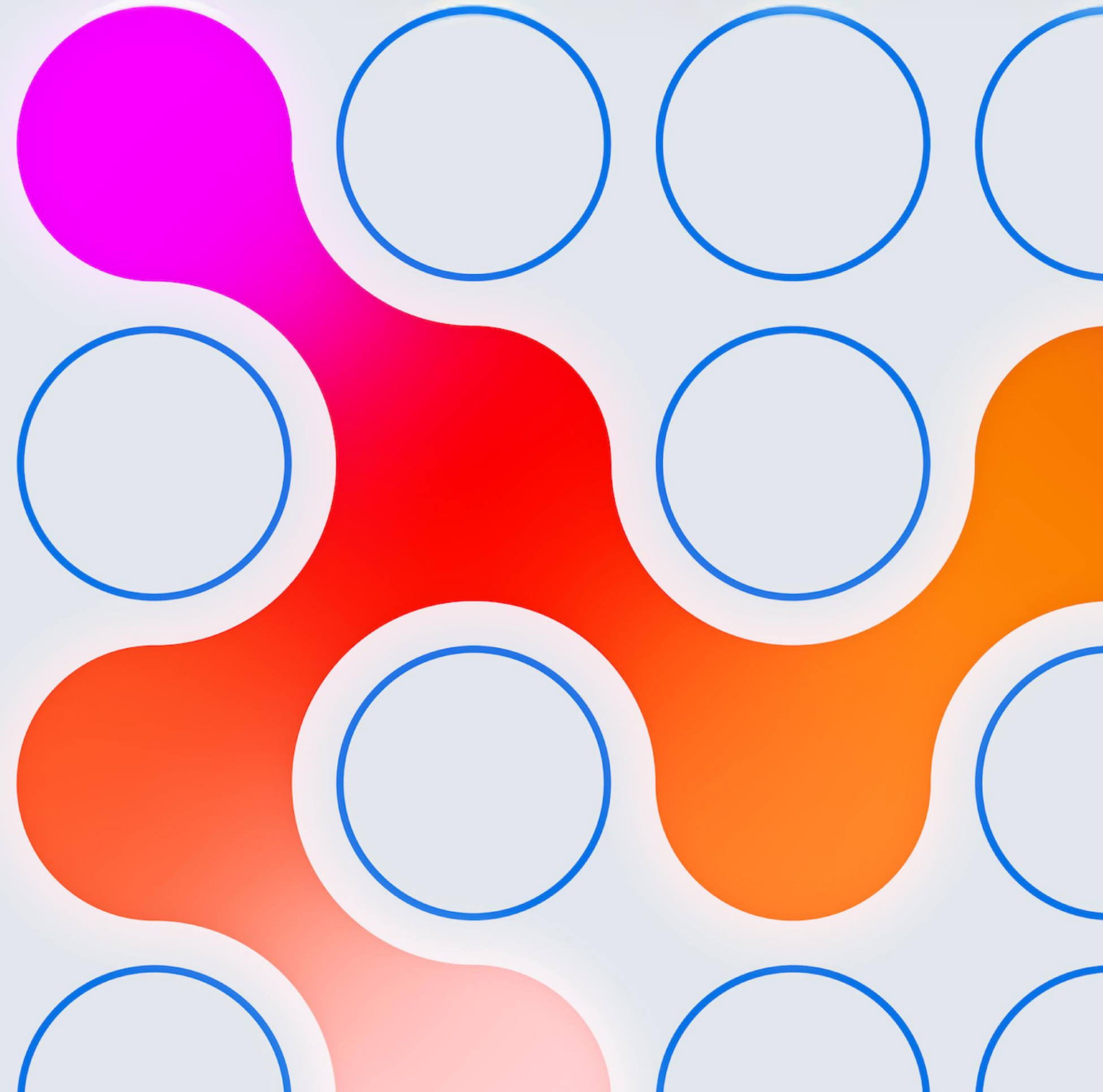
Sequential execution is unnecessarily slow. Independent subtasks—security audit, cost estimate, compliance check—should run in parallel.

Fault isolation

A single-agent failure derails the entire workflow. Multi-agent systems localize failures: One agent down does not stop the rest.

▶ **Rule of thumb:** Move to multi-agent when your single-agent workflow shows any two of these four triggers simultaneously.

Anatomy of a multi-agent system



The four planes of a multi-agent system

Control plane

Orchestration

The orchestrator decomposes tasks, delegates to subagents, monitors progress, and synthesizes results. It directs—it does not execute.

AWS:

Amazon Bedrock Multi-Agent Collaboration (supervisor mode) • AWS Step Functions

Execution plane

Specialized agents

Independent agents with scoped system prompts, curated tools, and purpose-built retrieval corpora. Independently deployable, evaluatable, and scalable.

AWS:

Amazon Bedrock AgentCore • AWS Lambda

State plane

Shared memory

The shared information layer: task state (durable), session context (low latency), and domain knowledge (retrieval). Agents coordinate through state, not direct coupling.

AWS:

Amazon DynamoDB • Amazon ElastiCache • Amazon Bedrock Knowledge Bases

Capability plane

Tools and MCP

Tools exposed through MCP servers—standardized, discoverable, and governed. Shared across agents with per-agent access controls at the server level.

AWS:

MCP Servers • Amazon Bedrock AgentCore • AWS Lambda (skill executors)

The DevOps Companion — Four-agent architecture

Orchestrator agent

Amazon Bedrock Multi-Agent Collaboration

Repo analysis

Tools:

GitHub MCP: list_files, read_file, search_code

Output:

ServiceManifest JSON

Infra generation

Tools:

CDK MCP: synthesize_stack, validate_template

Output:

InfraPackage (CDK app)

CI/CD config

Tools:

CodePipeline MCP: create_pipeline_definition

Output:

PipelineConfig JSON

Deploy and observe

Tools:

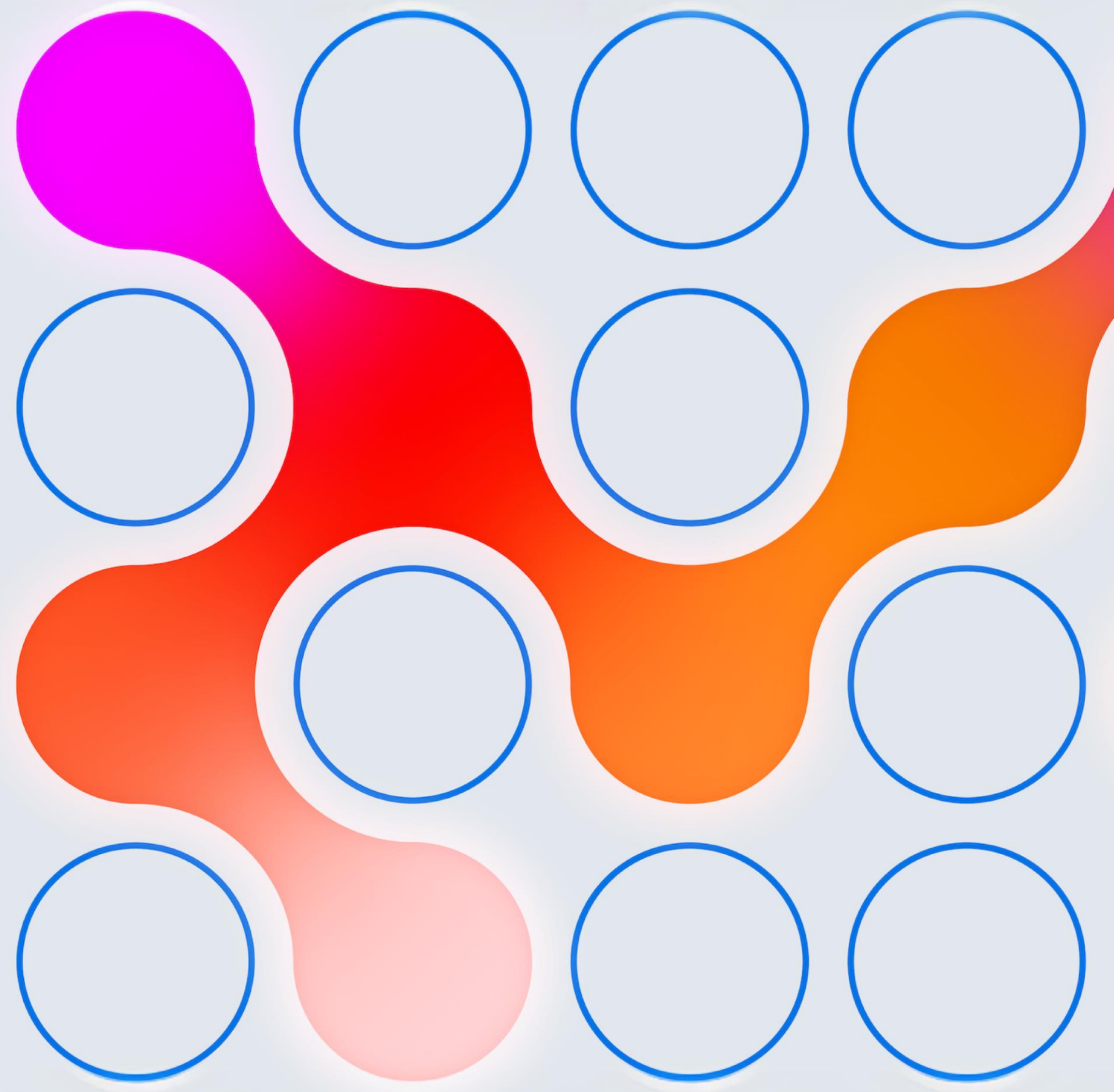
EKS MCP: deploy_workload; CloudWatch MCP: alarms

Output:

DeploymentStatus + ObsReport

▶ **Shared State:** DynamoDB (task state) · ElastiCache (session context) · Amazon Bedrock Knowledge Bases (domain knowledge)

Orchestration patterns: how agents coordinate



The four orchestration patterns

01 Centralized orchestration

When: Complex, open-ended tasks requiring dynamic planning and real-time adaptation

AWS implementation

Amazon Bedrock Multi-Agent (supervisor mode)

Watch out

Orchestrator is a single point of failure: Invest heavily here.

02 Skill-based dispatch

When: Fixed set of well-defined, parameterized operations invoked by a reasoning agent

AWS implementation

AWS Lambda functions behind MCP server interfaces

Watch out

Less flexible for open-ended tasks: Best for known, stable operations.

03 Handoff chains

When: Linear sequential pipelines where each step depends on the previous output

AWS implementation

Step Functions state machines with Choice states for branching

Watch out

Rigidity: Branching logic can become unwieldy without proper Choice state design.

04 Parallel fan-out and synthesis

When: Multi-domain tasks where independent specialist analysis can be parallelized

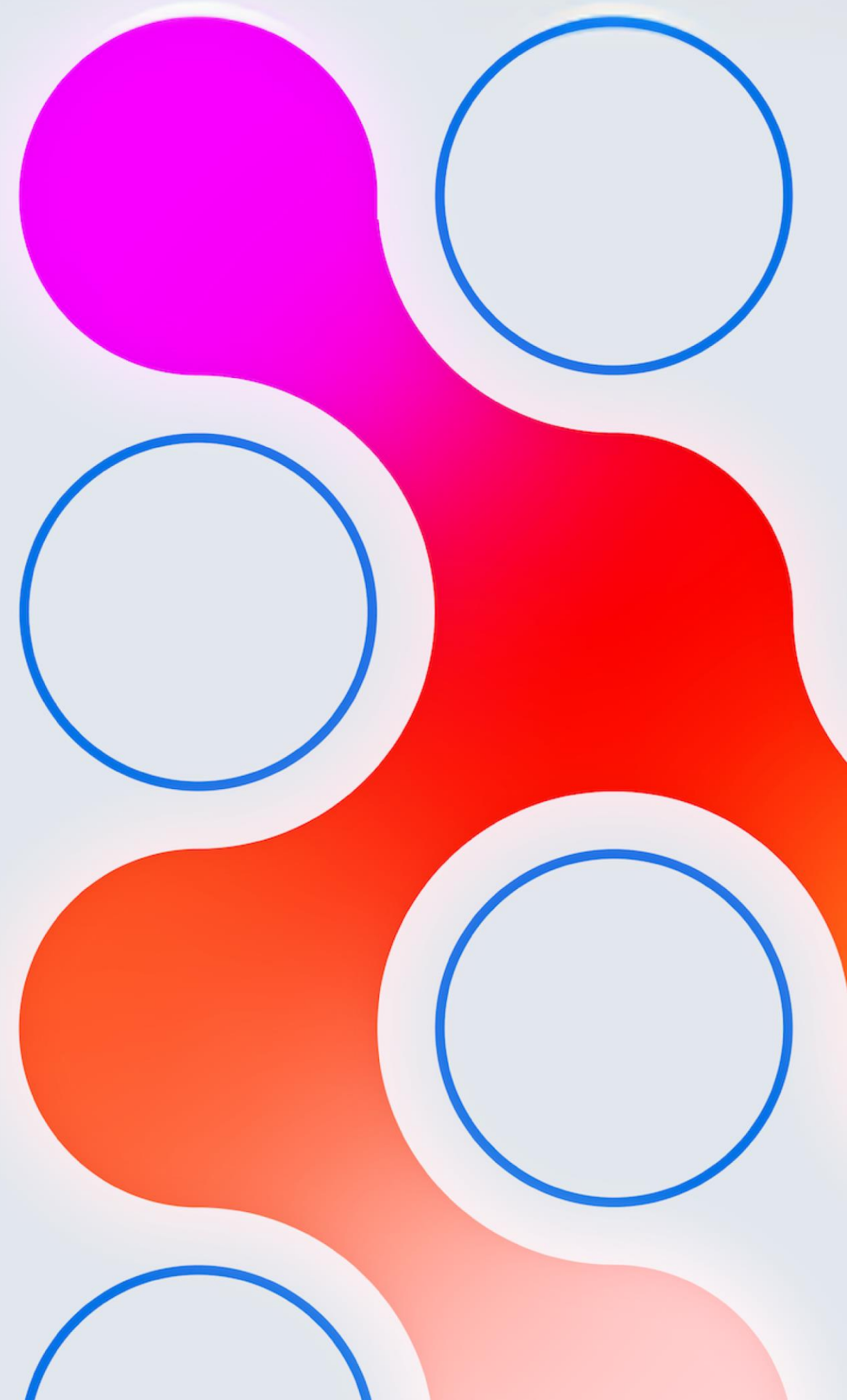
AWS implementation

Amazon EventBridge + Amazon SQS + Lambda specialists

Watch out

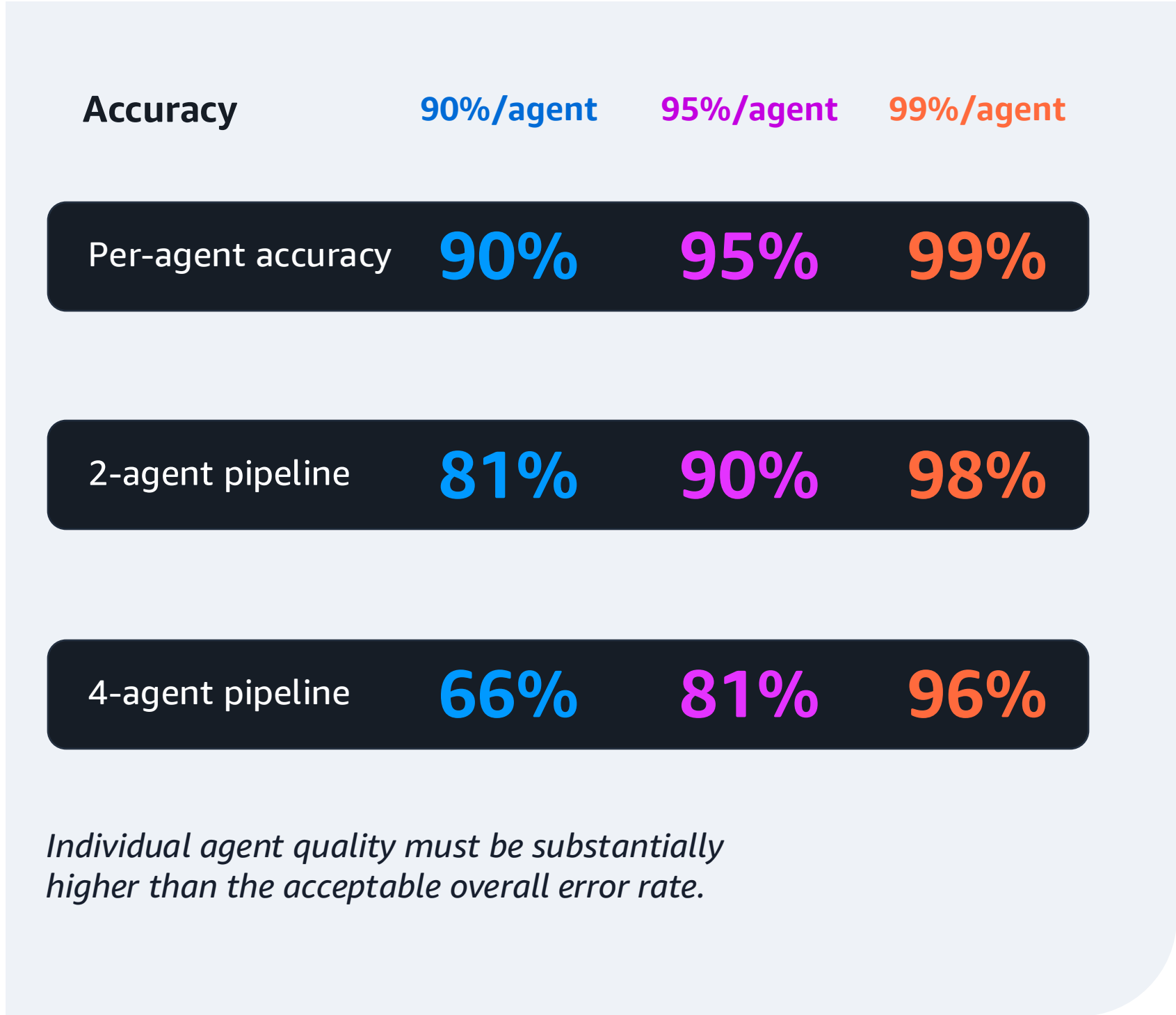
Synthesis step is routinely underbuilt: Handle conflicting results explicitly.

Non-determinism at scale: managing compounding uncertainty



Non-determinism compounds across agent boundaries

The compounding math



Defenses against propagation

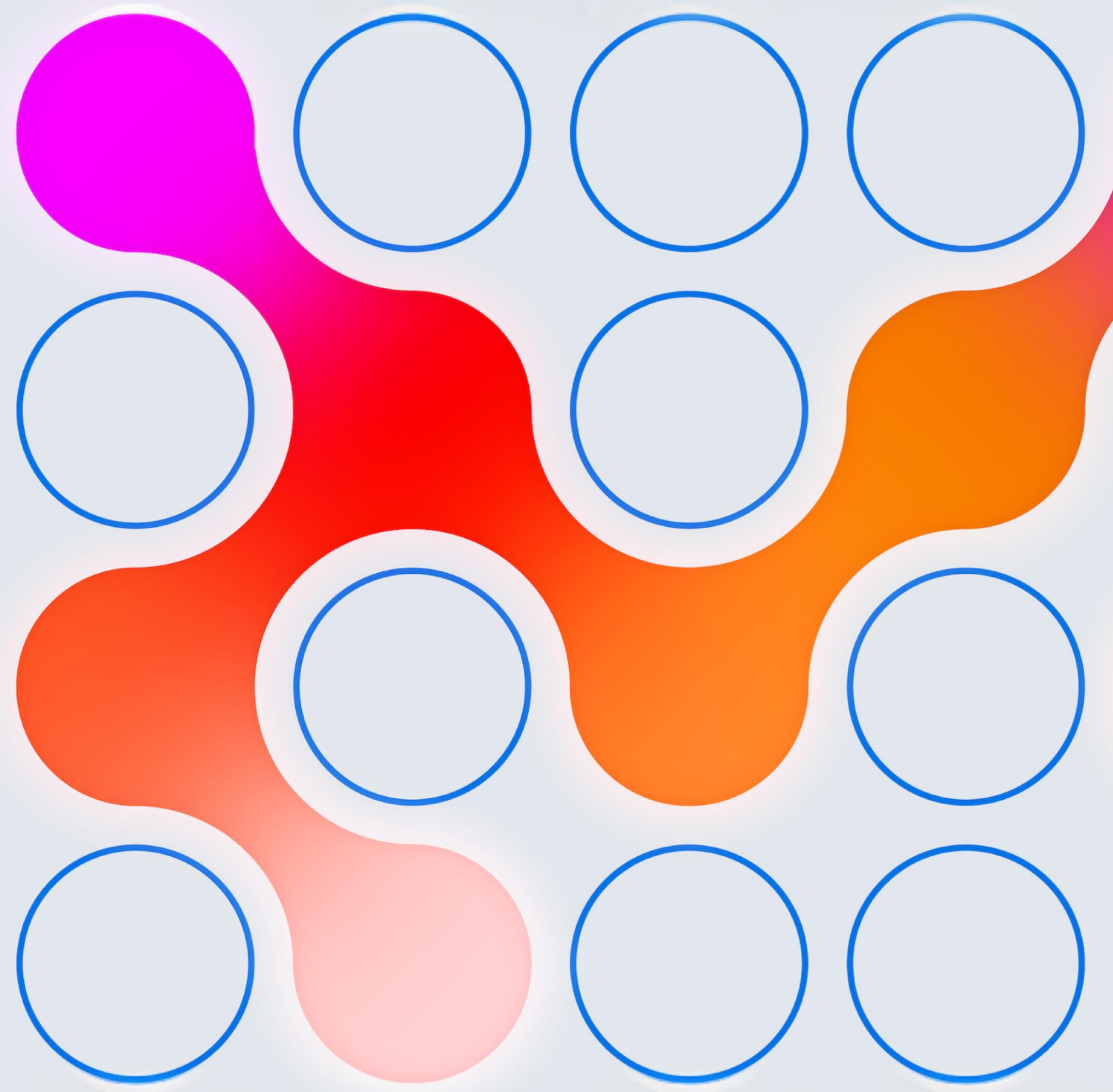
- Structured schema at every boundary**

Require JSON-schema-validated outputs at every handoff. Free-form inter-agent prose is a bug. Amazon Bedrock's structured output + Claude's native JSON constraints enable this without sacrificing reasoning quality.
- Abstention as a feature**

Agents uncertain about a task should return a structured abstention response — not a low-confidence guess. Build and evaluate abstention behavior explicitly in your golden datasets.
- End-to-end evaluation (non-optional)**

Per-agent evaluations are necessary but not sufficient. Run complete workflow evaluations. Amazon Bedrock AgentCore Evaluations Builtin. GoalSuccessRate measures system-level success, not per-agent output quality.

Shared context and knowledge exchange



What to Store Where and Why It Matters

Task state

Durable

Amazon DynamoDB

Single-digit ms reads · survives agent failures · audit trail for every workflow execution

Session context

Short-lived

Amazon ElastiCache

Sub-ms access · TTL-managed · conversation turns + recently accessed payloads

Domain knowledge

Read-heavy

Amazon Bedrock Knowledge Bases

Vector retrieval · governance-controlled · shared across all agents with no duplication

Intermediate results

Archival

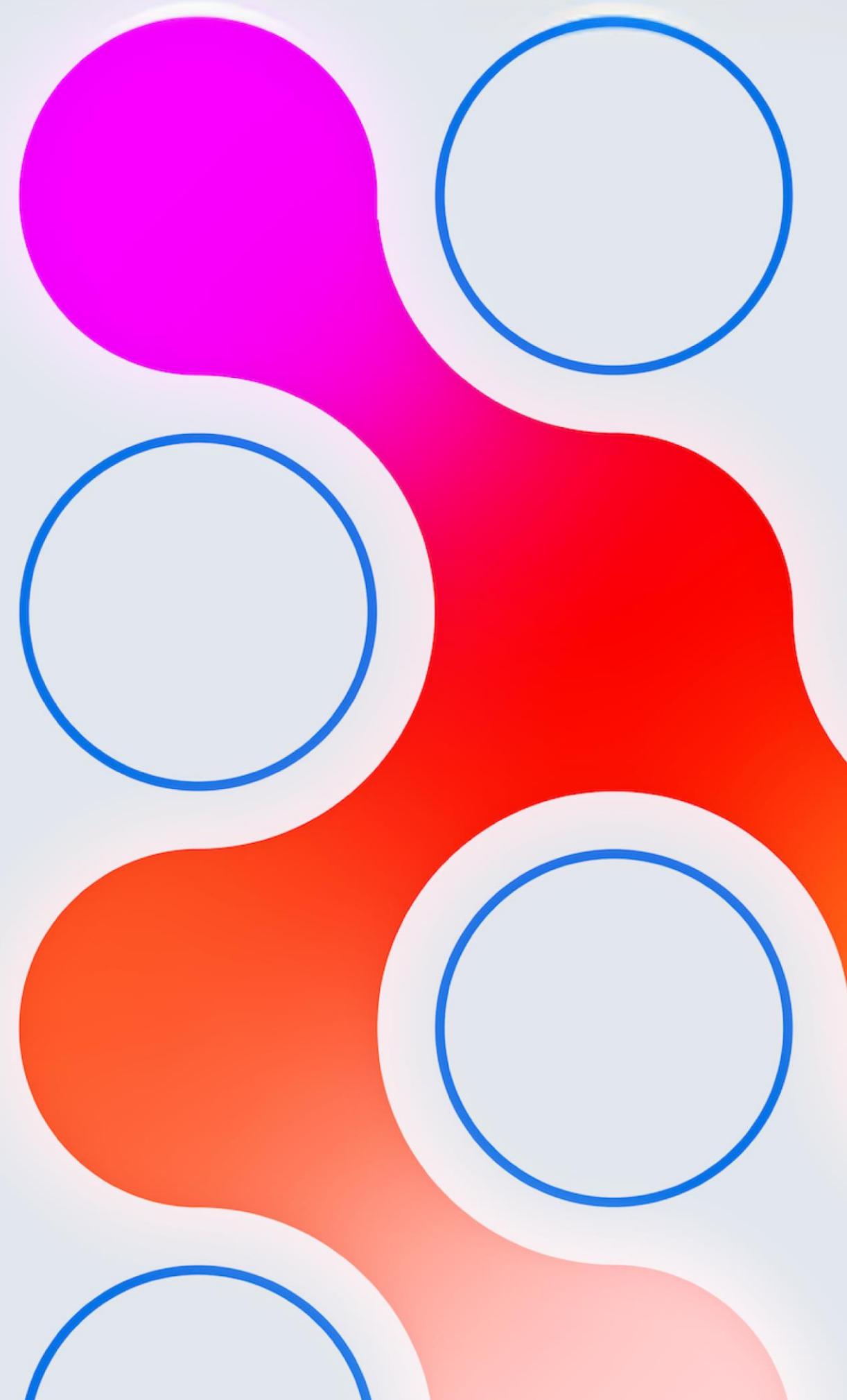
Amazon S3

Cost-effective · durable · referenced by task state record in DynamoDB, not forwarded in payloads



Handoff payload principle: Include what the next agent needs—nothing more. Verbose payloads bloat context windows and degrade reasoning quality.

MCP and A2A protocols — standards for capability and communication



MCP and A2A: Different problems, different protocols

MCP — Model Context Protocol

Agent → Tool interface

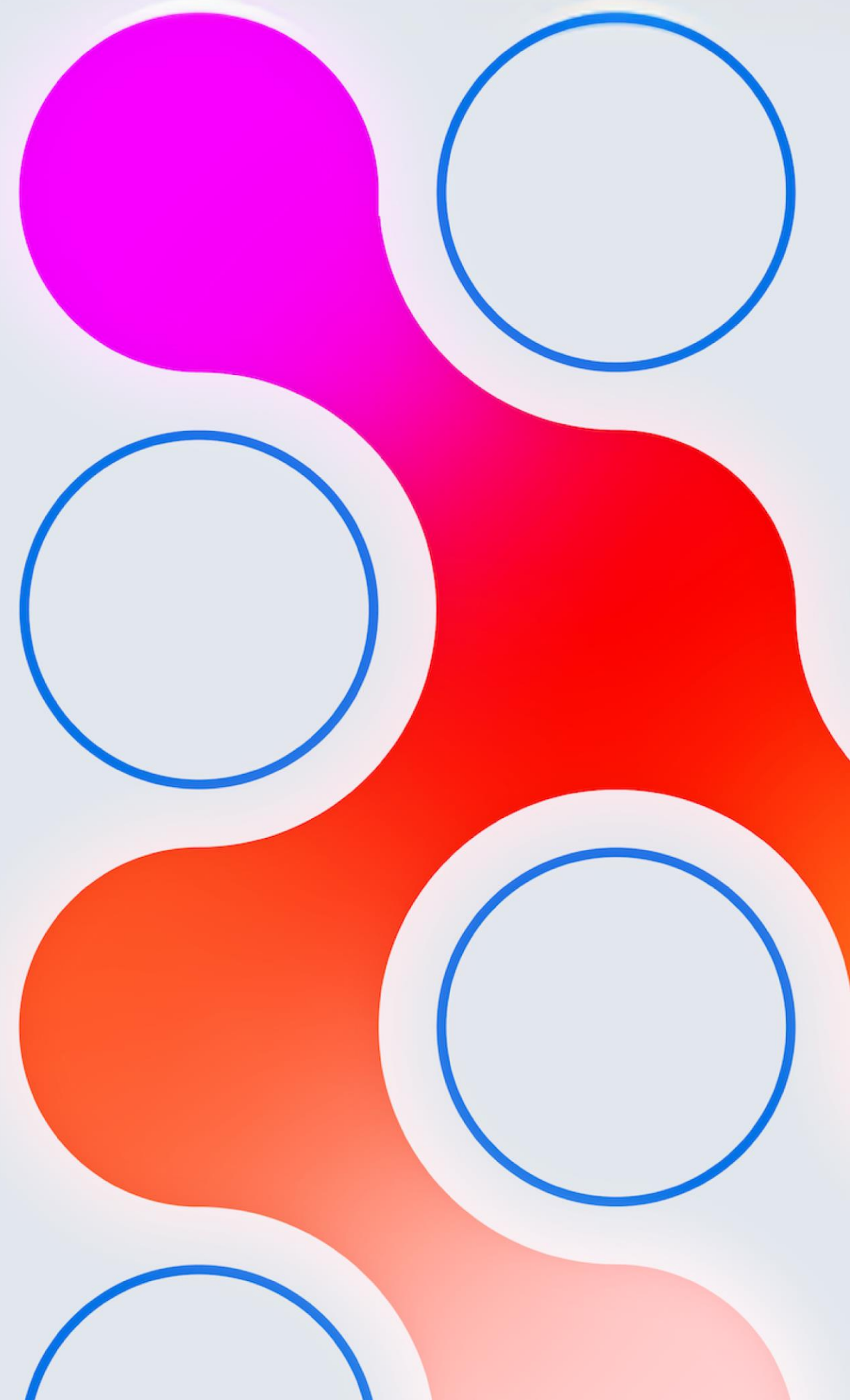
- Client-server protocol: agent is always the client, tool server is always the responder
- Provides standardized, discoverable, schema-validated tool interfaces
- Multi-agent MCP: server must implement per-agent access controls at the server level — not in system prompts
- Tool schema quality directly determines agent behavior quality
- Version-controlled: track MCP server version in each agent's config; test before updating in prod

A2A — Agent-to-Agent Protocol

Agent → Agent interface

- Peer protocol: any agent can initiate or receive—symmetric, unlike MCP's client-server model
- Task objects carry: description, context, authorized resources, expected output schema, constraints
- Agent cards: structured capability declarations at known endpoints—enable dynamic discoverability without pre-built integration
- A2A expands the attack surface: validate task objects against schema; apply Amazon Bedrock Guardrails at every A2A boundary
- Do not use MCP for agent-to-agent delegation—it is the wrong abstraction

Trust, identity and security in multi-agent systems

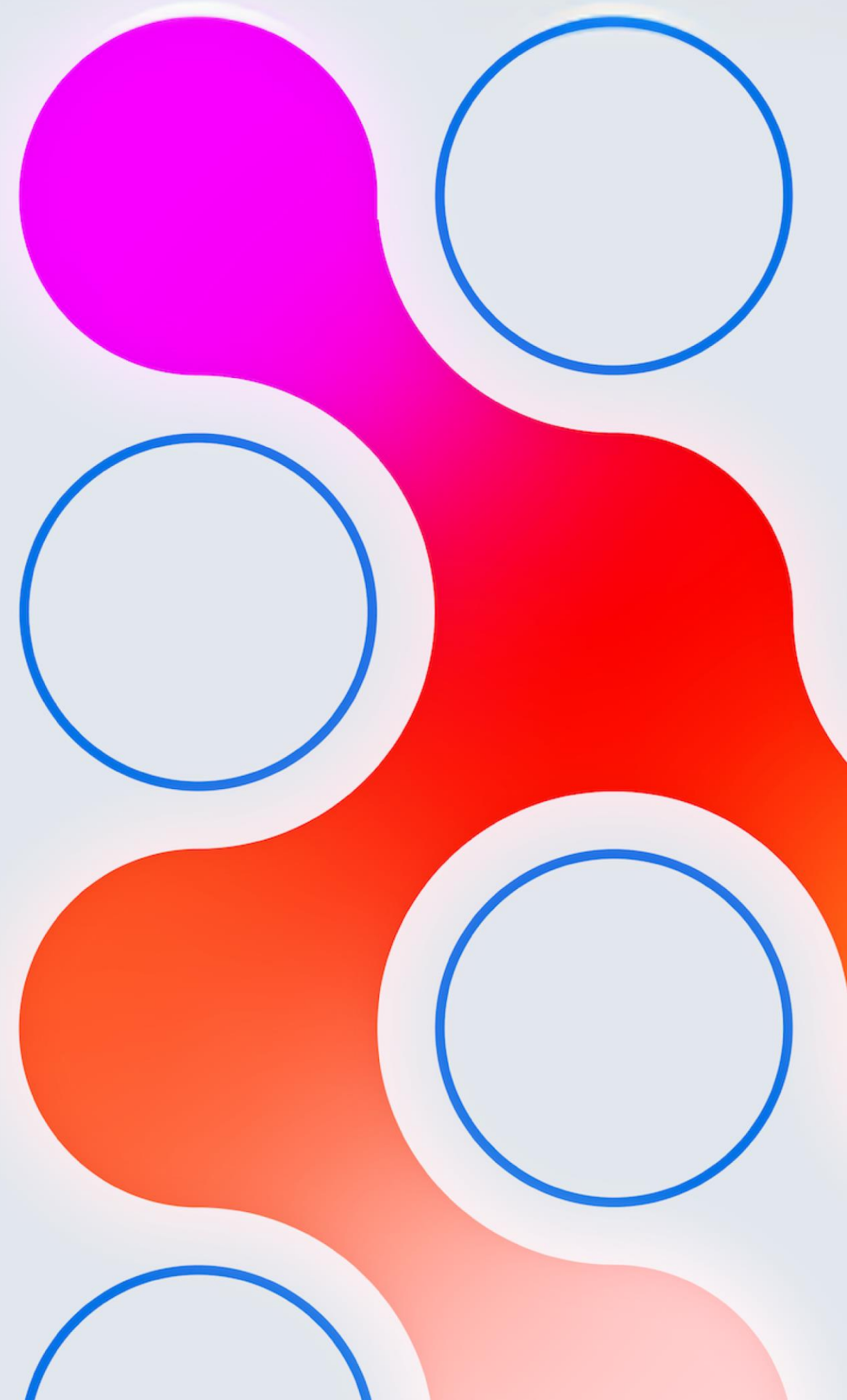


Security risks and mitigations at every agent boundary

Overprivileged subagents	Mitigation: Least-privilege scoped credentials per task. AWS Security Token Service (STS) generates time-limited credentials. CyberArk Agent Guard (AWS Marketplace) manages dynamic secret rotation.
Prompt injection propagation	Mitigation: Content filtering at every agent boundary via Amazon Bedrock Guardrails. Treat all externally-originated content as untrusted regardless of how many agents have processed it.
Conflicting agent outputs	Mitigation: Structured output validation and orchestrator conflict resolution logic. For safety-critical domains: the conservative result always wins.
Credential exposure in payloads	Mitigation: Never pass raw credentials in task objects. Use IAM role-based delegation. AWS IAM Roles Anywhere for non-AWS workloads.
Unauthorized agent discovery	Mitigation: Agent card registry with authentication requirements. Ping Identity (AWS Marketplace) handles identity orchestration for multi-tenant A2A deployments.

 **Zero trust:** Treat every agent boundary as a trust boundary. Apply security controls redundantly at MCP server, agent boundary, and orchestration layer.

Resilience, failure modes and distributed observability



Failure modes unique to multi-agent systems

Cascading failures

One failing agent takes down the entire chain. Circuit breakers in Step Functions catch repeated failures and route to fallback paths. Letting the rest of the system continue with degraded capability rather than total failure.

Temporal Technologies (AWS Marketplace): workflow resumes from exactly where it stopped after recovery—no re-execution of completed steps.

Orchestration loops

Orchestrators that repeatedly delegate tasks in a cycle run up unbounded costs. Every workflow needs explicit step counters, loop detection, and forced termination conditions.

Step Functions enforces maximum execution time and step limits natively. Set these from day one.

Conflicting parallel outputs

Security and operations agents evaluate the same configuration and reach opposite conclusions. The synthesis step must resolve conflicts explicitly — not with a generic summarize prompt.

Safety-critical domains: the conservative result always wins, regardless of how many agents disagree.

Context corruption at handoffs

Malformed, truncated, or semantically inconsistent payloads at handoff boundaries cause silent failures. Schema-validate every incoming handoff payload before acting — return a structured error to the orchestrator on validation failure.

Never proceed with a potentially corrupt payload. Return a structured error and let the orchestrator decide.

Distributed observability across agent boundaries

Why distributed tracing is non-negotiable

When a user reports incorrect infrastructure code, you need to answer:

Was it the analysis agent that produced a wrong service manifest?

The infra generation agent that misinterpreted a correct manifest?

The orchestrator that made a poor delegation decision?

Without distributed tracing, you are guessing. A consistent trace ID—propagated through every agent and tool call—is the only way to correlate logs into a single execution graph.

Observability tooling

Amazon CloudWatch traces

Distributed tracing across Lambda, Amazon Bedrock agent invocations, and Step Functions. Renders the full execution graph as a single navigable trace.

LangSmith

Purpose-built LLM tracing—hierarchical tree view from orchestrator down to individual tool calls. Evaluation dataset management for per-agent and multi-agent testing.

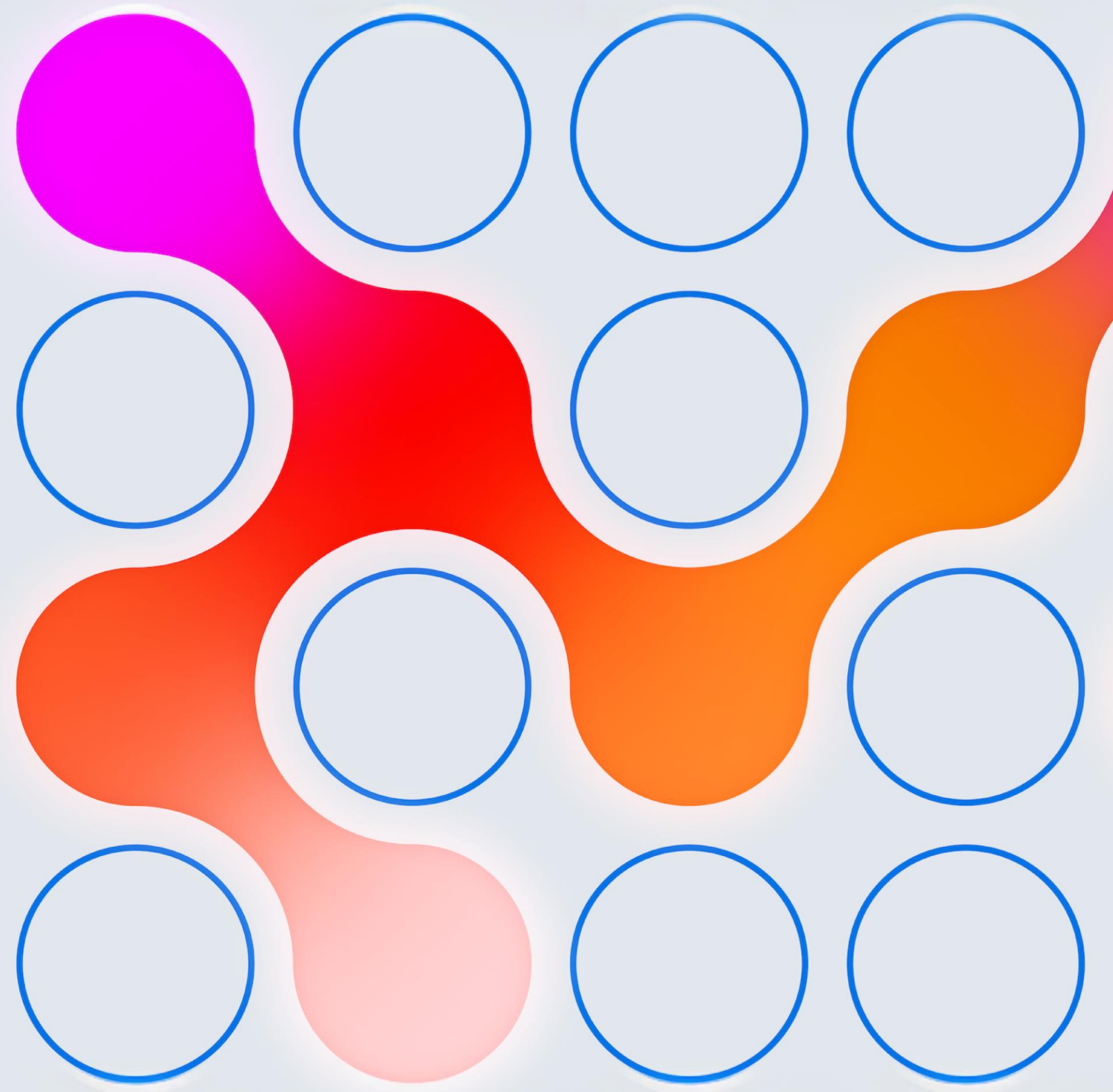
LangFuse

Token cost tracking across agents—shows which agent is responsible for the majority of inference costs. Strong cost attribution for optimization decisions.

Layered evaluation strategy

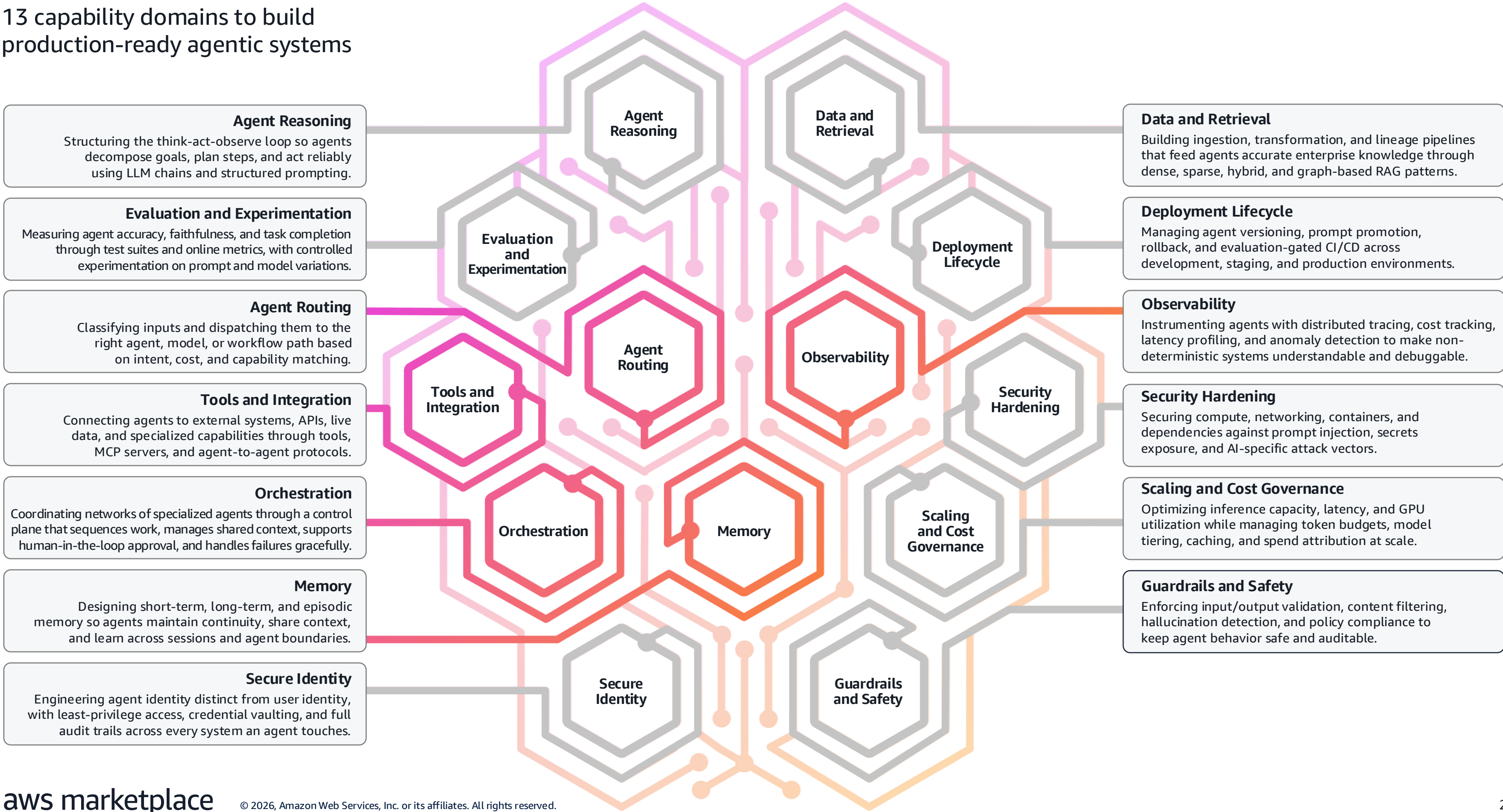
Per-agent (isolated datasets) + full workflow (Builtin.GoalSuccessRate) + differential analysis correlating per-agent scores against system-level outcomes.

Demo



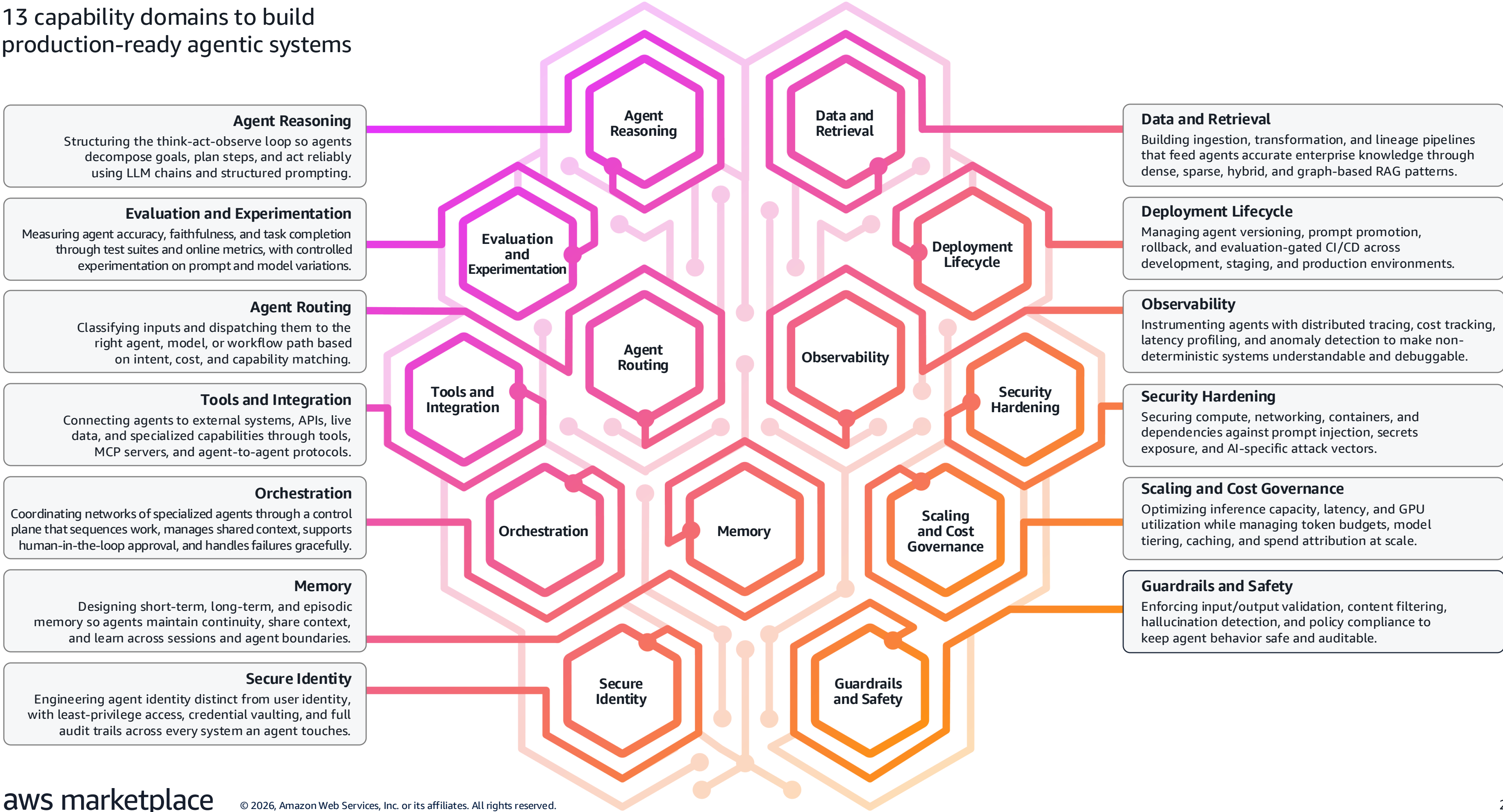
Recap: The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



Next steps

01

Define agent boundaries and responsibilities for your target workflow — document the four planes before writing any code.

02

Choose your orchestration pattern based on task structure: centralized for open-ended, handoff chain for linear, fan-out for parallel domains.

03

Design and version your handoff schemas. Register schemas before wiring agents together — schema-first is non-negotiable.

04

Set up DynamoDB task state table and ElastiCache session cache. Build shared state from day one — retrofitting is painful.

05

Enable distributed tracing with Amazon CloudWatch Traces and LangSmith/LangFuse. Build dashboards before you need to debug.

06

Implement least-privilege scoped credential delegation via AWS STS. Never pass orchestrator credentials to subagents.



Up next — Module 5: Domain-Specific Agent Applications — specialized agents for customer engagement, geospatial reasoning, and voice-enabled interaction, deployed on the multi-agent platform we just built.

Agentic AI tools and capabilities in AWS Marketplace

AGENT REASONING

Claude

cohere

Hugging Face

LangChain

Dify

Progress Agentic RAG

composable

Amazon Bedrock

STRANDS AGENTS

EVALUATION & EXPERIMENTATION

LangChain

Langfuse

vellum

Weights & Biases

comet LLM

Patronus AI

deepchecks.

Amazon Bedrock AgentCore (Evaluations)

Amazon SageMaker AI

Amazon CloudWatch

AGENT ROUTING

LangChain

AWS Step Functions

Amazon EventBridge

Amazon Bedrock

TOOLS & INTEGRATION

algolia

mapbox

Deepgram

Kong

Konnect

Tyk

TAILWINDS

portkey

SMARTBEAR.

krakenD

AWS Lambda

Amazon API Gateway

Amazon Bedrock AgentCore (Gateway)

ORCHESTRATION

Temporal

orkes

PREFECT

ASTRONJMER

AWS Step Functions

Amazon Bedrock Agents

Amazon MWA

MEMORY

zilliz

redis

elastic

TiDB Cloud

MongoDB Atlas

Weaviate

yugabyteDB

Pinecone

Amazon OpenSearch

Amazon MemoryDB

Amazon Bedrock AgentCore (Memory)

SECURE IDENTITY

Ping Identity

okta

CYBERARK

Amazon Cognito

AWS Secrets Manager

Amazon Bedrock AgentCore (Identity)

DATA & RETRIEVAL

CONFLUENT

databricks

Fivetran

Collibra

Alation

Bigeye

MONTE CARLO

boomi

Amazon Bedrock Knowledge Bases

Amazon S3

AWS Glue

DEPLOYMENT LIFECYCLE

Pulumi

GitLab

Vercel

LaunchDarkly

Vault

KNIME

AWS CodePipeline

Amazon SageMaker AI

AWS CDK

OBSERVABILITY

DATADOG

honeycomb.io

arize

LangWatch

True Watch

INSTANA

logz.io

Amazon CloudWatch

AWS X-Ray

AWS Distro for OpenTelemetry

SECURITY HARDENING

snyk

Protect AI

Prompt

tailscale

Amazon GuardDuty

AWS Security Hub

Amazon Inspector

SCALING & COST GOVERNANCE

ClickHouse

NVIDIA AI ENTERPRISE

Modal

cast.ai

baseten

anyscale

hydrolix

StormForge

Amazon EKS

AWS Fargate

Amazon Bedrock AgentCore (Runtime)

GUARDRAILS & SAFETY

Claude

Guardrails AI

Arthur

Amazon Bedrock Guardrails

Amazon Bedrock AgentCore (Policy)

2,800+

Agentic AI tools and capabilities

To help you build, deploy, and operate agentic systems on AWS — available through your AWS account.



© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

29

Why AWS Marketplace?

Streamline product discovery, procurement, and deployment.



AWS Marketplace offers pricing flexibility

Flexible consumption models

Ease into commitments

Free trials

Get started with a free trial before committing to purchase.



Bring your own license

Migrate to AWS with your existing product licenses. Intended for pre-existing customers.



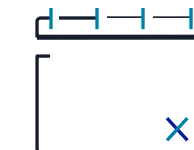
Usage-based subscriptions

Pay for software and compute capacity by the second or hour with no long-term commitments.



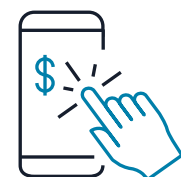
Contracts

Make a monthly, annual, or multi-year payment and receive a discount.



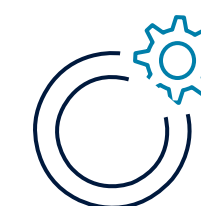
Private offers

Negotiate a custom price with a seller or Channel Partner. Accept the offer in AWS Marketplace.



Flexible payment scheduler

Set a custom billing frequency with your seller.



Next steps



Multi-agent architectures on AWS

Build production multi-agent pipelines with LangGraph, AWS Step Functions, Amazon EventBridge, and Amazon MQ — from supervisor patterns to reliable inter-agent messaging.



Building agentic systems on AWS

Explore the 14-module workshop series. Learn to design, orchestrate, and scale multi-agent systems—from foundational patterns to production deployment—with structured tutorials and vetted AWS Marketplace integrations.

Thank you!

